

PES University — Dept. of CSE
Lab 1: Requirements Engineering & UML Use-Case Modelling
Problem Statement #44 — Database Query Performance Profiler

PES2UG24CS483

SHREYA NARAYANA BHAT

Primary Domain	Developer Tools & IT Operations
Target Actors	Database Administrator, Backend Lead, System Scheduler
System Summary	A database observability tool that ingests slow query logs, parses SQL execution plans, identifies missing index candidates, and generates weekly performance optimization digests.

1. Requirements Table

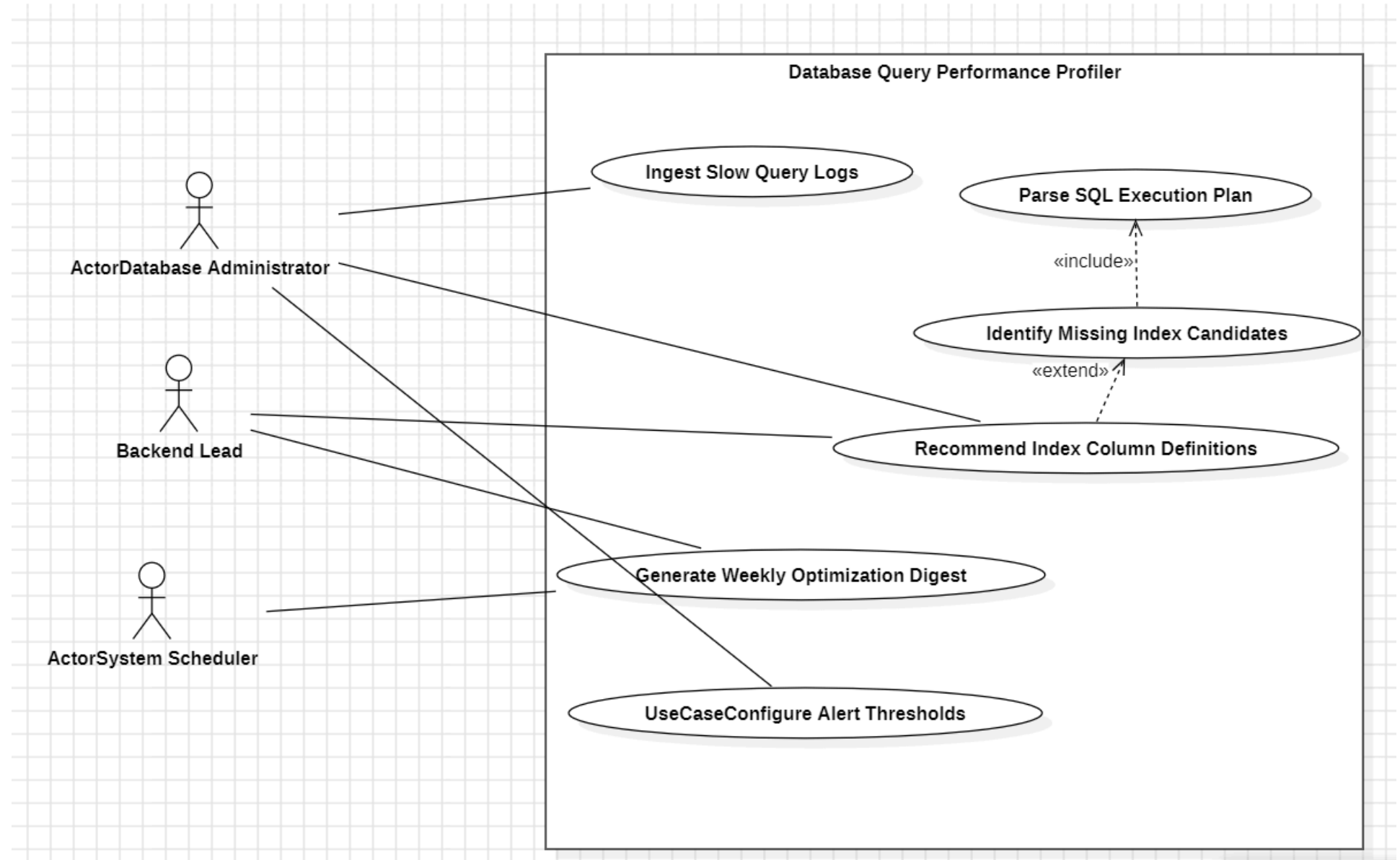
Rows shaded in blue are the sample requirements provided in the problem statement. FR-002–FR-005 and NFR-002 were authored for this system.

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Functional	The system shall parse Postgres/MySQL EXPLAIN plans, highlight sequential table scans on large tables, and recommend optimal index column definitions.	High	Pass: Missing composite index identified on filtered columns. Fail: Query plan with sequential scan marked optimized.	Core capability that turns raw execution plans into actionable index guidance.
FR-002	Functional	The system shall detect and flag missing composite indexes by analyzing the filtered, joined, and sorted columns referenced in ingested slow query logs.	High	Pass: A query filtering on 2+ columns without a matching composite index is flagged with the suggested column order. Fail: Flagged query already has a covering composite index.	Composite-index gaps are a leading cause of slow queries and are not always obvious from a single EXPLAIN plan.
FR-003	Functional	The system shall generate a weekly performance optimization digest summarizing the top 10 slowest queries, missing index candidates, and estimated query-cost savings.	High	Pass: Digest is generated every 7 days and lists exactly the top 10 queries ranked by cumulative execution time. Fail: Digest omits a query in the top 10 or is not produced within the scheduled window.	Gives the Backend Lead a periodic, digestible summary instead of requiring manual log review.
FR-004	Functional	The system shall allow the Database Administrator to configure alert thresholds (e.g., query execution time, log ingestion volume) that trigger real-time notifications when exceeded.	Medium	Pass: A query exceeding the configured threshold triggers a notification within 60 seconds. Fail: No notification is sent when a configured threshold is breached.	Enables proactive intervention before a slow query degrades production performance.

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-005	Functional	The system shall allow the Backend Lead to view the weekly digest on a dashboard and export it as a PDF or CSV report.	Medium	Pass: Exported PDF/CSV contains the same query list, index recommendations, and metrics shown on the dashboard. Fail: Exported file is missing data present on the dashboard.	Supports sharing performance findings with stakeholders outside the tool.
NFR-001	Nonfunctional	Slow query log ingestion must handle up to 5,000 log records per minute without dropping entries or slowing the host database.	High	Pass: Benchmarking tests confirm target throughput and security standards under simulated peak load. Fail: Any entry is dropped or host DB latency increases beyond baseline during the test.	Ensures the profiler scales to production log volumes without becoming a bottleneck itself.
NFR-002	Nonfunctional	The system shall encrypt slow query log data at rest (AES-256) and in transit (TLS 1.2 or higher) to protect sensitive query text and table metadata.	High	Pass: Security scan confirms all stored log data is encrypted at rest and all network transfers use TLS 1.2+. Fail: Any log data is found stored or transmitted unencrypted.	Slow query logs may contain sensitive table/column names and query parameters that require protection.

2. UML Use-Case Diagram

Actors: Database Administrator, Backend Lead, System Scheduler (automated actor that triggers the weekly digest job).



Relationships modelled

- «include» — UC-03 Identify Missing Index Candidates always includes UC-02 Parse SQL Execution Plan, since a plan must be parsed before missing indexes can be identified.
- «extend» — UC-04 Recommend Index Column Definitions extends UC-03 Identify Missing Index Candidates, since a recommendation is only produced when a missing-index condition is actually found (optional/conditional behaviour).

Use cases

- UC-01 Ingest Slow Query Logs — Database Administrator
- UC-02 Parse SQL Execution Plan — system-internal, triggered via «include»
- UC-03 Identify Missing Index Candidates — Database Administrator
- UC-04 Recommend Index Column Definitions — Database Administrator, Backend Lead
- UC-05 Generate Weekly Optimization Digest — Backend Lead, System Scheduler
- UC-06 Configure Alert Thresholds — Database Administrator

3. Use-Case Flow Specification

Use Case: UC-03 — Identify Missing Index Candidates

Preconditions

- At least one slow query log batch has been successfully ingested (UC-01 complete).
- The Database Administrator is authenticated and has access to the query profiler dashboard.

Postconditions

- Success: One or more missing composite index candidates are recorded against the offending query, each with a suggested column order and estimated cost savings.
- Failure: No index candidates are recorded and the query is marked as 'reviewed — no index gap found'.

Main Success Scenario

1. Database Administrator selects a slow query (or batch of queries) from the ingested log to analyze.
2. System retrieves the corresponding EXPLAIN execution plan for the selected query (UC-02, «include»).
3. System parses the execution plan and identifies sequential/full table scans on tables above the configured size threshold.
4. System cross-references the filtered, joined, and sorted columns in the query against existing indexes on the affected table(s).
5. System determines that no existing index covers the column combination and flags a missing composite index candidate.
6. System estimates the query-cost reduction if the candidate index were created and attaches this estimate to the finding.
7. System displays the missing index candidate, with suggested column order, on the Database Administrator's dashboard.
8. Database Administrator reviews the candidate and marks it as accepted for follow-up.
9. Use case ends successfully.

Alternate Flow

5a. No Missing Index Found

- a. 5a1. System determines that an existing index already covers the filtered/joined columns.
- b. 5a2. System marks the query as 'reviewed — no index gap found' and does not raise a candidate.
- c. 5a3. Use case ends; no «extend» to UC-04 Recommend Index Column Definitions occurs, since that use case only triggers when a candidate is actually found.